

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO



TÀI LIỆU THIẾT KẾ HỆ THỐNG

SkinSight
Ứng dụng phân tích làn da

Nhóm [Title Card]

1. Trần Xuân Bảo - 23020332
2. Phạm Quốc Hùng - 23020373
3. Phan Tuấn Hiệp - 23020364
4. Nguyễn Khánh Tùng - 23020434
5. Phan Hoàng Dũng - 23020346

HÀ NỘI, 05/2025

Lời tựa

Sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI) và thị giác máy tính (Computer Vision) đã mở ra nhiều khả năng ứng dụng trong lĩnh vực chăm sóc sức khỏe và sắc đẹp, đặc biệt là trong phân tích tình trạng da mặt từ hình ảnh. Việc xây dựng một hệ thống AI hoạt động chính xác, ổn định và có thể triển khai trong thực tế đòi hỏi sự phối hợp chặt chẽ giữa nhiều thành phần kỹ thuật khác nhau - từ thiết kế kiến trúc hệ thống, triển khai mô hình học sâu, đến quản lý dữ liệu và tối ưu trải nghiệm tích hợp.

Tài liệu thiết kế hệ thống này được biên soạn nhằm phục vụ như một bản mô tả kỹ thuật toàn diện cho dự án AI Skinsight. Tài liệu cung cấp cái nhìn rõ ràng về kiến trúc tổng thể, thiết kế các module chức năng, cách tổ chức mã nguồn, cấu trúc API, cũng như các lựa chọn công nghệ và quy tắc thiết kế được áp dụng xuyên suốt dự án.

Với tài liệu này, nhóm phát triển có thể thống nhất phương pháp triển khai, nhóm kiểm thử có thể dễ dàng xây dựng quy trình kiểm định chất lượng, và các bên liên quan kỹ thuật có thể nhanh chóng nắm bắt cấu trúc hệ thống để phục vụ cho việc tích hợp, mở rộng hoặc đánh giá.

Hy vọng tài liệu này sẽ là cơ sở vững chắc, góp phần đảm bảo chất lượng và hiệu quả trong suốt quá trình phát triển của hệ thống.

Phiên bản tài liệu

Phiên bản	Ngày	Mô tả
1.0	16/05/2025	Phiên bản đầu tiên, bao gồm phần lời tựa và giới thiệu của tài liệu.
1.1	20/05/2025	Cập nhật các yêu cầu hệ thống, kiến trúc tổng thể và thiết kế các mô hình AI.
1.2	25/05/2025	Cập nhật thiết kế thành phần, hướng phát triển và kết luận, sửa lỗi ngữ pháp. Phiên bản hoàn thiện của tài liệu.

Mục lục

1. Giới thiệu	6
1.1. Mục tiêu hệ thống	6
1.2. Phạm vi hệ thống	6
1.3. Đối tượng sử dụng tài liệu	7
2. Yêu cầu hệ thống	8
2.1. Yêu cầu chức năng	8
2.1.1. User stories	8
2.1.2. Biểu đồ ca sử dụng	9
2.1.3. Biểu đồ tuần tự	13
2.1.4. Biểu đồ lớp	15
2.2. Yêu cầu phi chức năng	16
2.2.1. Hiệu năng	16
2.2.2. Bảo mật	16
2.2.3. Khả năng sử dụng	16
2.2.4. Khả năng bảo trì và mở rộng	16
2.2.5. Tính sẵn sàng và phục hồi	17
3. Kiến trúc tổng thể	18
3.1. Các thành phần chính	18
3.2. Luồng hoạt động tổng quan	19
4. Thiết kế thành phần	21
4.1. Thiết kế API	21
4.2. Thiết kế cơ sở dữ liệu	23
4.3. Thiết kế giao diện người dùng	23
5. Thiết kế chi tiết mô hình AI	26
5.1. Mô hình phát hiện mụn	26
5.1.1. Kiến trúc mô hình	26
5.1.2. Dữ liệu huấn luyện	27
5.1.3. Cấu hình huấn luyện	27
5.2. Mô hình phát hiện nám	28
5.2.1. Kiến trúc mô hình	28
5.2.2. Dữ liệu huấn luyện	28
5.2.3. Cấu hình huấn luyện	29
5.3. Mô hình phát hiện nếp nhăn	29
5.3.1. Kiến trúc mô hình	29

5.3.2. Dữ liệu huấn luyện	30
5.3.3. Cấu hình huấn luyện	30
5.4. Chuyển đổi mô hình sang định dạng ONNX	31
6. Hướng phát triển trong tương lai	32
7. Kết luận	33

1. Giới thiệu

1.1. Mục tiêu hệ thống

AI Skinsight là một hệ thống phân tích ảnh khuôn mặt ứng dụng trí tuệ nhân tạo, được thiết kế nhằm hỗ trợ đánh giá ba đặc điểm quan trọng của làn da gồm: tình trạng mụn, sự xuất hiện của các vết nám và nếp nhăn. Mục tiêu chính của hệ thống là cung cấp một giải pháp tự động, nhanh chóng và chính xác cho người dùng có nhu cầu theo dõi và phân tích tình trạng da mặt của họ thông qua ảnh chụp khuôn mặt.

Hệ thống cho phép người dùng tải ảnh khuôn mặt lên thông qua giao diện ứng dụng hoặc dịch vụ API, sau đó sẽ tự động kiểm tra ảnh đầu vào, phát hiện chính xác khuôn mặt, và chạy qua ba mô hình được huấn luyện chuyên biệt để phân tích từng vấn đề da liễu nêu trên. Kết quả phân tích sẽ được tổng hợp và trả về, cho phép tích hợp dễ dàng vào các hệ thống chăm sóc da, spa, hoặc các ứng dụng tư vấn da liễu trực tuyến.

AI Skinsight được thiết kế như một nền tảng backend độc lập, tập trung vào xử lý ảnh và cung cấp kết quả phân tích, không bao gồm giao diện người dùng cuối. Phần frontend sẽ được phát triển tách biệt và kết nối thông qua các API mà hệ thống cung cấp.

1.2. Phạm vi hệ thống

Hệ thống AI Skinsight tập trung vào các chức năng chính: tiếp nhận và xác thực ảnh đầu vào từ người dùng, phân tích ảnh khuôn mặt bằng mô hình AI để nhận diện tình trạng mụn, nám và nếp nhăn, và cuối cùng là trả kết quả phân tích thông qua REST API. Ngoài ra, hệ thống cũng cung cấp một số API hỗ trợ như kiểm tra trạng thái hoạt động của server và xử lý lỗi đầu vào không hợp lệ.

Phạm vi triển khai hệ thống bao gồm các thành phần chính như module kiểm tra số lượng khuôn mặt trong ảnh, ba mô hình AI xử lý song song cho từng loại vấn đề da, hệ thống API xử lý yêu cầu và phản hồi, cùng với lớp tiền xử lý dữ liệu ảnh. Hệ thống không lưu trữ ảnh người dùng sau khi xử lý nhằm đảm bảo tính riêng tư và tuân thủ các nguyên tắc bảo vệ dữ liệu cá nhân. Ngoài ra, AI Skinsight không cung cấp chẩn đoán y khoa, mà chỉ đóng vai trò như một công cụ phân tích hỗ trợ, với kết quả mang tính gợi ý tham khảo.

1.3. Đối tượng sử dụng tài liệu

Tài liệu thiết kế này được xây dựng với mục tiêu phục vụ nhiều đối tượng kỹ thuật khác nhau trong quá trình phát triển, kiểm thử và triển khai hệ thống.

- **Nhà phát triển hệ thống (backend, AI):** nắm rõ kiến trúc, API, cách triển khai và tích hợp các model AI.
- **Kỹ sư DevOps:** hiểu cách triển khai hệ thống, cấu hình Docker, giám sát hệ thống.
- **Tester / QA:** có căn cứ để viết test case, đánh giá chất lượng đầu ra.
- **Nhóm frontend / tích hợp:** nắm rõ giao diện API để gọi phân tích ảnh.
- **Khách hàng kỹ thuật / chuyên gia kỹ thuật bên thứ ba:** tham khảo kiến trúc và cách hoạt động để đánh giá hoặc mở rộng.

2. Yêu cầu hệ thống

2.1. Yêu cầu chức năng

Hệ thống AI Skinsight được thiết kế để phục vụ hai nhóm người dùng chính: người dùng cuối (User) và quản trị viên hệ thống (Admin). Mỗi nhóm có các chức năng và quyền hạn riêng, đảm bảo khả năng truy cập phù hợp với vai trò trong hệ thống. Phần này trình bày các chức năng chính của hệ thống dưới dạng mô tả chi tiết kèm theo các user story và các loại biểu đồ tương ứng.

2.1.1. User stories

Để đảm bảo hệ thống đáp ứng đúng nhu cầu thực tế của người dùng và quản trị viên, các yêu cầu chức năng được mô hình hóa dưới dạng các user stories sau, phản ánh các tình huống sử dụng cụ thể và mục tiêu mà mỗi nhóm người dùng muốn đạt được trong quá trình tương tác với hệ thống.

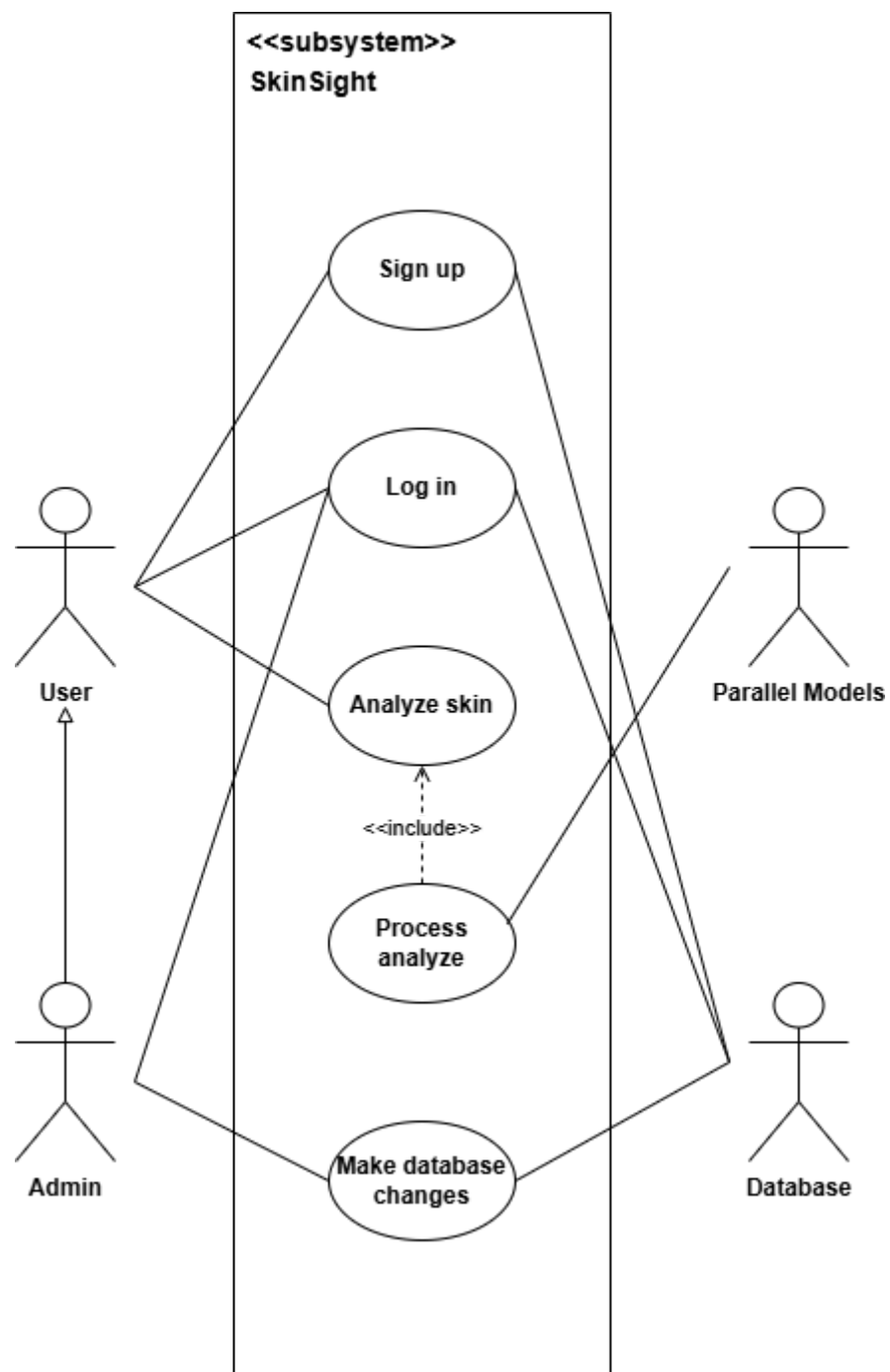
- **Cả sử dụng chung**
 - **Đăng ký tài khoản**
Là một người dùng mới, tôi muốn có thể **đăng ký tài khoản** bằng email và mật khẩu để truy cập hệ thống và sử dụng dịch vụ phân tích ảnh da mặt.
 - **Đăng nhập hệ thống**
Là một người dùng đã đăng ký, tôi muốn có thể **đăng nhập vào hệ thống** để sử dụng các chức năng của hệ thống.
- **Đối với người dùng cuối (User)**
 - **Xem thông tin cá nhân**
Là một người dùng, tôi muốn có thể **xem thông tin hồ sơ** của mình để theo dõi thông tin tài khoản và trạng thái đăng nhập.
 - **Chỉnh sửa hồ sơ cá nhân**
Là một người dùng, tôi muốn **chỉnh sửa thông tin cá nhân** như tên, mật khẩu hoặc ảnh đại diện để đảm bảo thông tin được cập nhật chính xác.
 - **Phân tích ảnh theo yêu cầu (Request-based)**
Là một người dùng, tôi muốn **tải ảnh khuôn mặt** lên hệ thống để nhận được kết quả phân tích về mụn, nám và nếp nhăn trên da.

- **Phân tích ảnh thời gian thực (Real-time)**
Là một người dùng, tôi muốn hệ thống hỗ trợ **phân tích ảnh trực tiếp từ camera** để xem kết quả phân tích da mặt một cách nhanh chóng và tiện lợi.
- **Đối với quản trị viên hệ thống (Admin)**
 - **Xem danh sách người dùng**
Là một quản trị viên, tôi muốn có thể **xem danh sách tất cả người dùng** trong hệ thống để theo dõi và quản lý hoạt động của họ.
 - **Thêm người dùng mới**
Là một quản trị viên, tôi muốn **thêm tài khoản người dùng mới** để cấp quyền sử dụng hệ thống.
 - **Xóa người dùng**
Là một quản trị viên, tôi muốn có thể **xóa người dùng khỏi hệ thống** khi họ không còn hoạt động hoặc vi phạm chính sách sử dụng.
 - **Cập nhật vai trò người dùng**
Là một quản trị viên, tôi muốn **cập nhật vai trò của người dùng** để phân quyền hợp lý và bảo đảm tính bảo mật hệ thống.

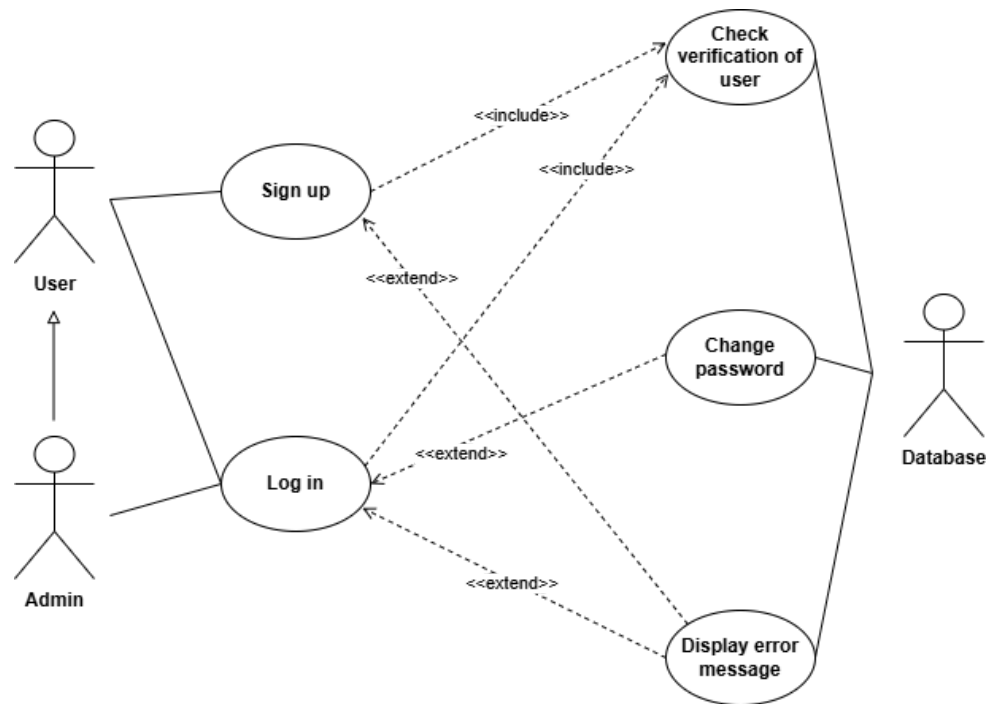
2.1.2. Biểu đồ ca sử dụng

Dựa trên các User Stories đã phân tích, ta xác định được các chức năng chính mà hệ thống cần đáp ứng cho từng loại người dùng. Từ đó, sơ đồ use case được xây dựng nhằm mô hình hóa mối quan hệ giữa các tác nhân (users) và các chức năng của hệ thống một cách trực quan và có hệ thống.

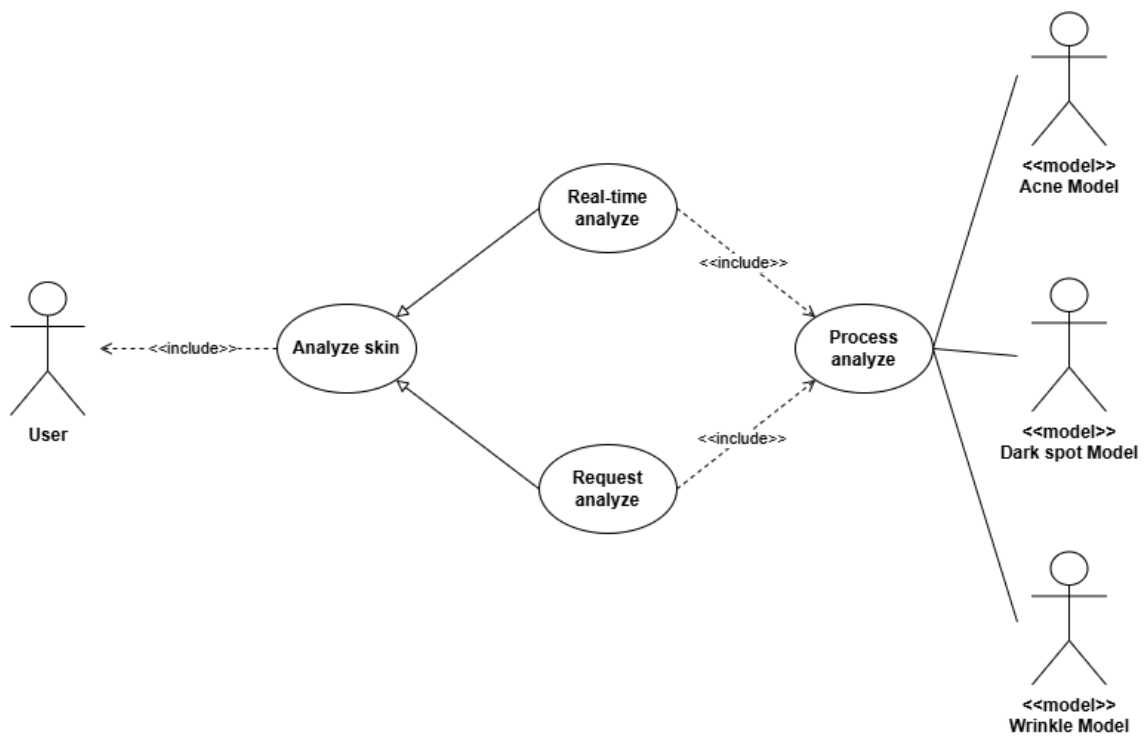
- Sơ đồ use case tổng quan



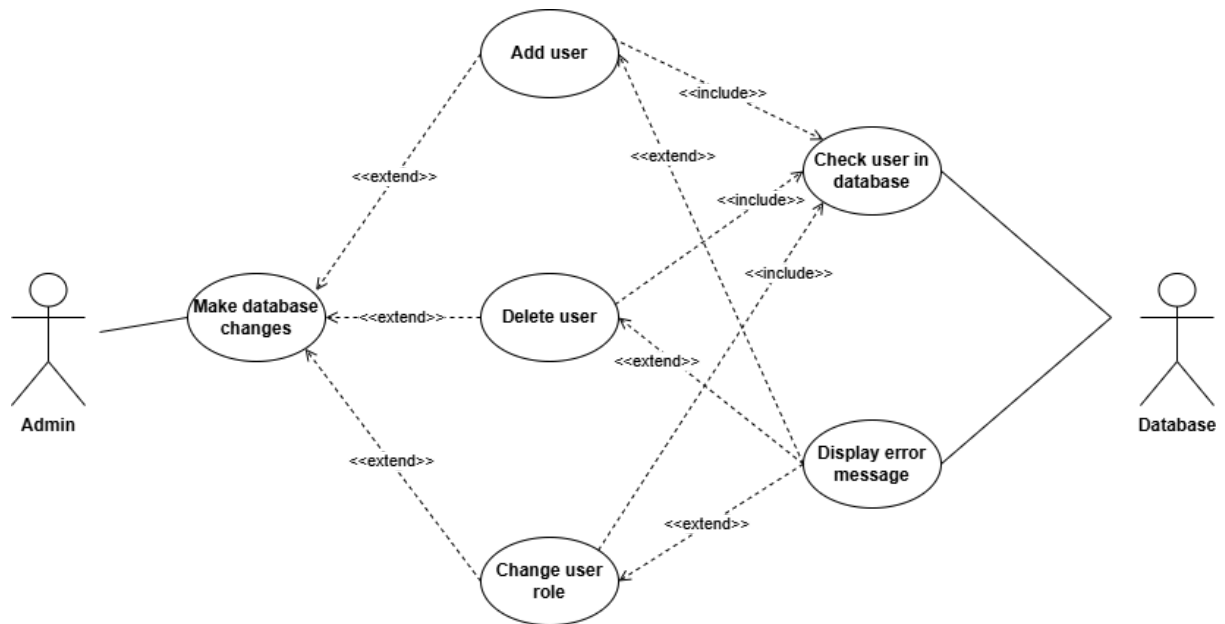
- Chức năng đăng ký và đăng nhập



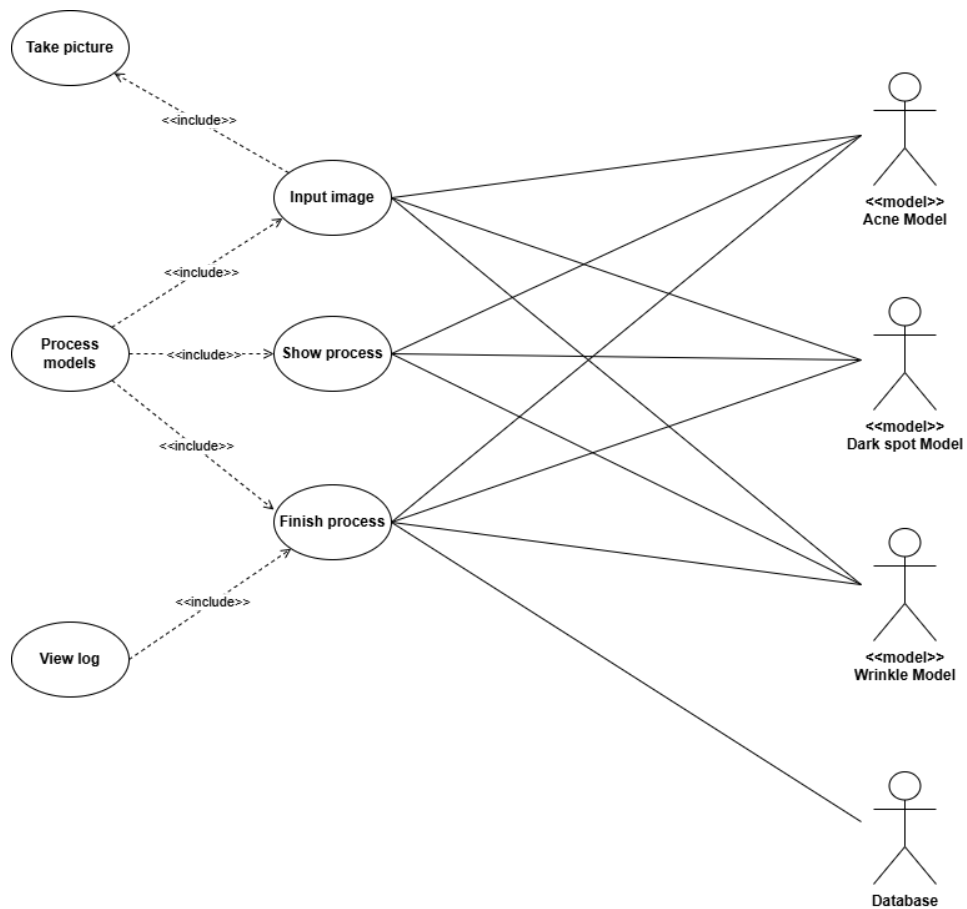
- Chức năng phân tích da



- Thay đổi cơ sở dữ liệu



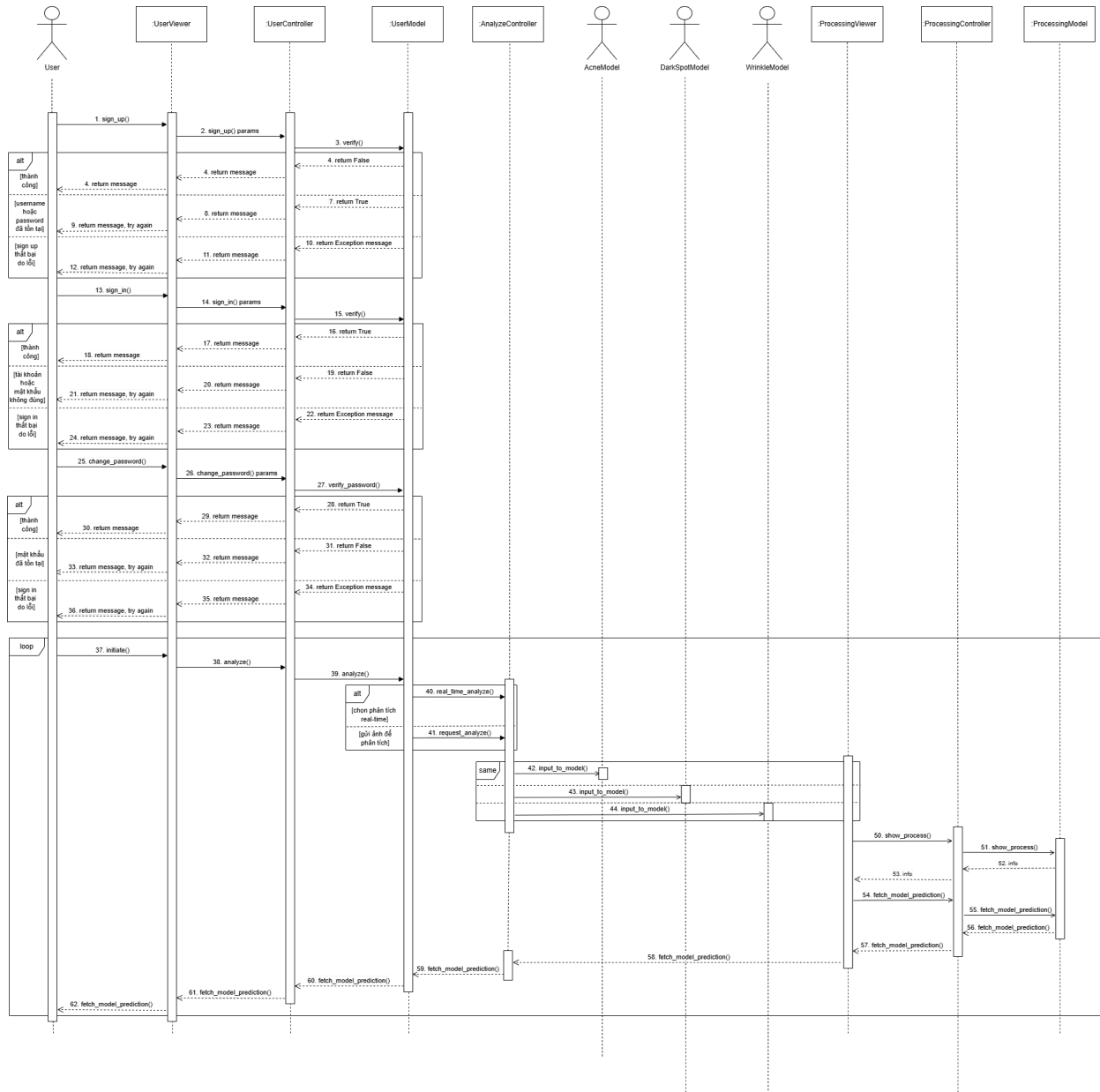
- Sử dụng các model AI



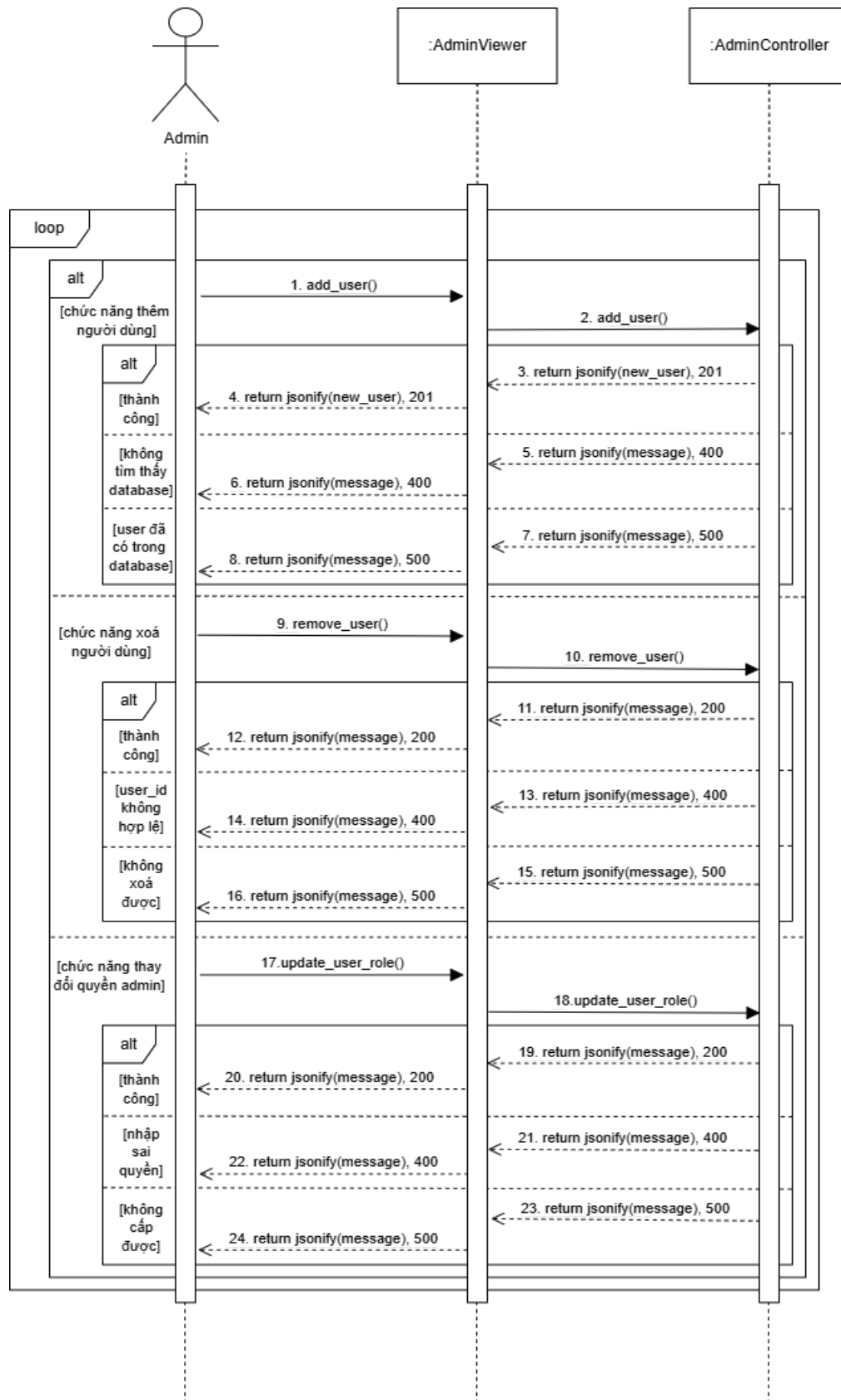
2.1.3. Biểu đồ tuần tự

Để làm rõ hơn cách các thành phần trong hệ thống phối hợp thực hiện các chức năng theo từng tình huống người dùng (user stories), nhóm sử dụng biểu đồ tuần tự. Các biểu đồ này giúp minh họa chi tiết luồng tương tác giữa người dùng, hệ thống và các mô-đun xử lý trong quá trình thực thi chức năng.

● Đối với người dùng (User)



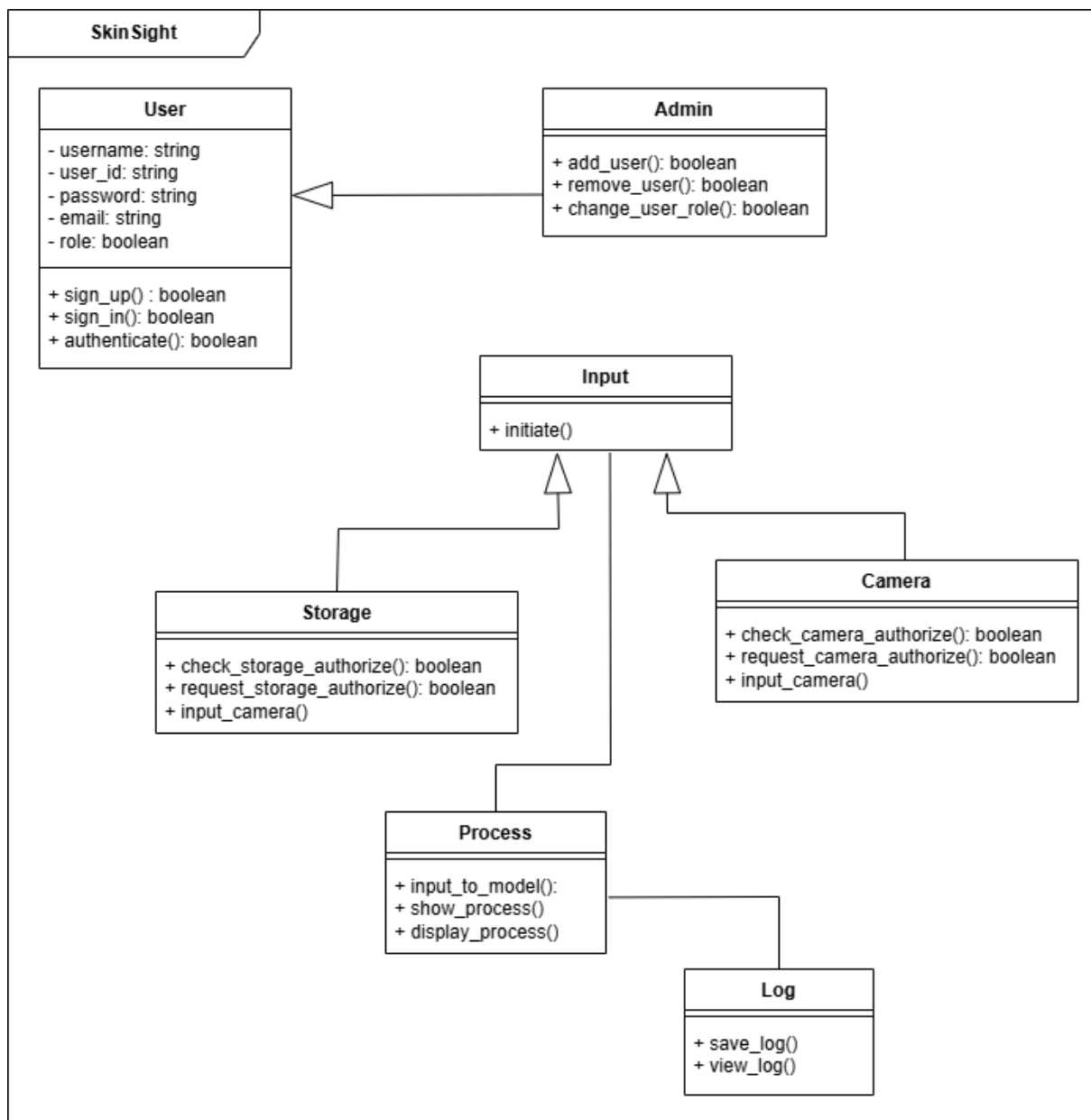
- Đối với quản trị viên hệ thống (Admin)



2.1.4. Biểu đồ lớp

Từ các biểu đồ tuần tự mô tả luồng tương tác và hành vi của hệ thống theo từng tình huống người dùng, chúng ta có thể xác định rõ các đối tượng, lớp và mối quan hệ giữa chúng. Điều này giúp xây dựng biểu đồ lớp, thể hiện cấu trúc tĩnh của hệ thống với các lớp, thuộc tính và phương thức, từ đó làm rõ cách tổ chức và thiết kế phần mềm một cách toàn diện hơn.

Biểu đồ lớp để khái quát hệ thống các lớp, hàm được sử dụng, các attribute và method của mỗi lớp:



2.2. Yêu cầu phi chức năng

Hệ thống AI Skinsight không chỉ cần thực hiện đúng chức năng mà còn phải đạt các tiêu chuẩn nhất định về hiệu suất, bảo mật, khả năng mở rộng và khả năng sử dụng. Các yêu cầu phi chức năng dưới đây nhằm đảm bảo hệ thống hoạt động hiệu quả, ổn định và an toàn khi phục vụ người dùng thật.

2.2.1. Hiệu năng

- Hệ thống phải xử lý ảnh và trả kết quả phân tích cho người dùng trong vòng dưới 5 giây trong điều kiện bình thường.
- Thời gian phản hồi API phải dưới 300 ms cho các thao tác không liên quan đến mô hình AI.
- Hệ thống phải hỗ trợ xử lý song song ảnh của nhiều người dùng cùng lúc, tối thiểu 50 phiên làm việc đồng thời.

2.2.2. Bảo mật

- Dữ liệu người dùng, bao gồm thông tin cá nhân và ảnh khuôn mặt, phải được truyền qua kết nối HTTPS để đảm bảo an toàn.
- Hệ thống phải hỗ trợ xác thực người dùng bằng token sau khi đăng nhập.
- Ảnh người dùng không được lưu trữ vĩnh viễn nếu không có sự đồng ý, và nếu có lưu, phải được mã hóa và bảo vệ quyền truy cập.
- Hệ thống phải giới hạn quyền truy cập vào chức năng quản trị, chỉ dành cho người dùng có vai trò admin.

2.2.3. Khả năng sử dụng

- Giao diện người dùng phải đơn giản, dễ hiểu và thân thiện, hỗ trợ cả trên máy tính và thiết bị di động.
- Người dùng không cần kiến thức kỹ thuật để sử dụng hệ thống: việc chụp ảnh, gửi ảnh và nhận kết quả phải thực hiện chỉ qua vài bước rõ ràng.
- Giao diện phản hồi phải hiển thị kết quả trực quan, dễ hiểu, đánh dấu hiển thị phân vùng da có vấn đề.

2.2.4. Khả năng bảo trì và mở rộng

- Hệ thống phải được thiết kế theo hướng mô-đun hóa (modular) để dễ dàng mở rộng thêm các mô hình AI mới hoặc thay đổi pipeline xử lý ảnh.
- Có khả năng triển khai theo dạng container (Docker) để dễ mở rộng quy mô xử lý bằng cách nhân bản model server khi lượng người dùng tăng.
- Mã nguồn backend và frontend phải được tổ chức theo mô hình MVC (Model - View - Controller) hoặc tương đương, giúp dễ bảo trì.
- Hệ thống phải ghi log đầy đủ mọi tương tác quan trọng để hỗ trợ việc kiểm tra và xử lý sự cố.

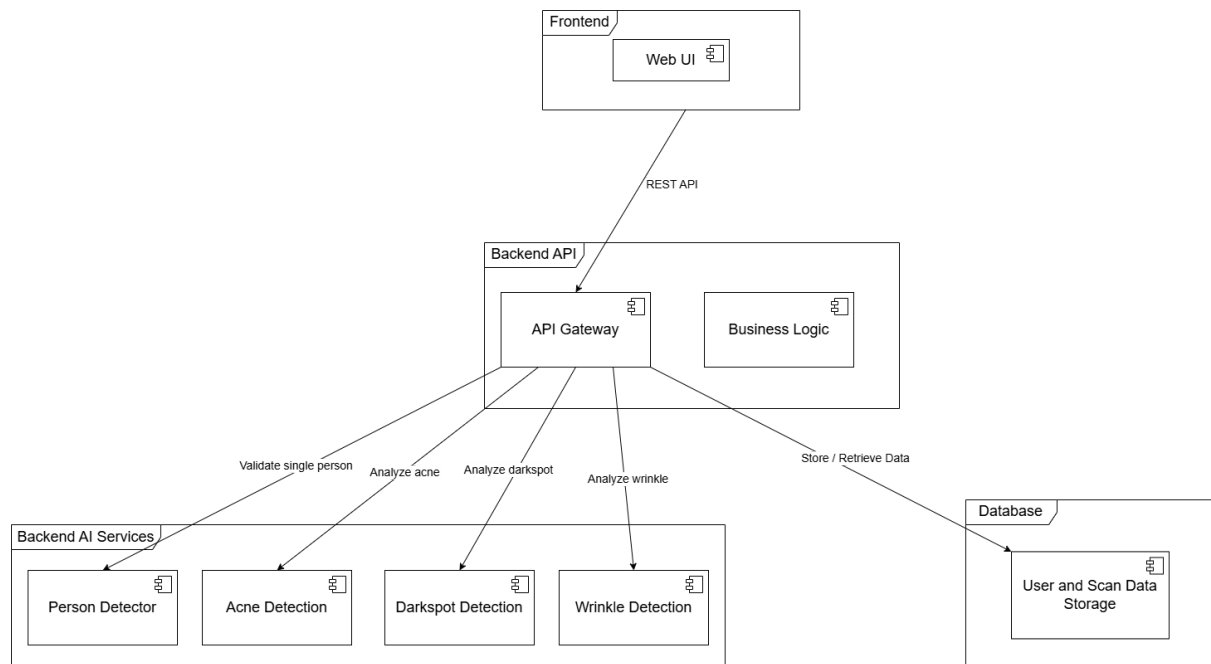
2.2.5. Tính sẵn sàng và phục hồi

- Hệ thống phải duy trì thời gian hoạt động trên 99% trong điều kiện triển khai ổn định.
- Nếu một mô hình AI gặp lỗi khi xử lý, hệ thống vẫn có thể trả kết quả từ các mô hình còn lại, kèm theo cảnh báo lỗi.
- Hệ thống cần cơ chế retry tự động đối với các tác vụ quan trọng như phân tích ảnh bị lỗi tạm thời.

3. Kiến trúc tổng thể

Dự án được xây dựng theo kiến trúc microservices với nhiều module nhỏ chuyên trách từng chức năng riêng biệt, được tổ chức thành các thư mục và container Docker độc lập. Điều này giúp tăng tính mô-đun, dễ bảo trì, mở rộng và triển khai trên các môi trường khác nhau.

Biểu đồ thành phần của hệ thống



3.1. Các thành phần chính

- **Backend AI Services**

Bao gồm các dịch vụ chuyên trách xử lý ảnh và phân tích da, được đóng gói riêng biệt trong các thư mục:

- **acne**: dịch vụ phát hiện mụn
- **darkspot**: dịch vụ phát hiện nám
- **wrinkle**: dịch vụ phát hiện nếp nhăn

Mỗi dịch vụ có **Dockerfile**, **app.py** làm **entrypoint**, cùng các file cấu hình và mô hình (**model.txt**), giúp dễ dàng phát triển, kiểm thử và triển khai độc lập.

- **Backend API**

Đóng vai trò trung gian quản lý toàn bộ các yêu cầu từ frontend, cung cấp RESTful API

- Các routes được chia theo chức năng (admin, auth, user).
- services chứa logic nghiệp vụ như xác thực, quản lý người dùng.
- controllers điều phối và xử lý tương tác giữa API và dịch vụ AI.
- utils chứa các tiện ích dùng chung.

- **Database**

Thành phần lưu trữ dữ liệu hệ thống, có Dockerfile và cấu hình riêng biệt để quản lý thông tin người dùng, kết quả quét, lịch sử, quyền truy cập, v.v.

- **Frontend**

Giao diện người dùng web, gồm các thư mục chứa tệp CSS, HTML, JavaScript.

- Hỗ trợ người dùng đăng nhập, xem lịch sử quét, tải ảnh lên để phân tích, và nhận kết quả.
- Frontend được cấu hình phục vụ qua Nginx (có file nginx.conf) và Dockerfile riêng để triển khai ổn định, bảo mật.

- **Docker Compose**

File *docker-compose.yml* quản lý toàn bộ các container dịch vụ, đảm bảo chạy đồng bộ backend API, backend AI modules, database, và frontend. Từ đó, dễ dàng cấu hình môi trường phát triển hoặc sản xuất với một lệnh duy nhất.

- **Tests và Tài liệu**

Thư mục tests chứa các bài kiểm thử nhằm đảm bảo chất lượng mã nguồn. Các file cấu hình và tài liệu như *.gitignore*, *README.md*, *requirements.txt* giúp quản lý dự án hiệu quả.

3.2. Luồng hoạt động tổng quan

Hệ thống hoạt động dựa trên luồng xử lý chính xoay quanh việc phân tích ảnh khuôn mặt người dùng để đánh giá tình trạng da. Quy trình này bao gồm các bước liên kết giữa frontend, backend, và các mô hình AI, được mô tả như sau:

- **Tương tác từ phía người dùng:**

Người dùng có thể chọn gửi ảnh trực tiếp từ thiết bị hoặc sử dụng webcam tích hợp trên giao diện frontend. Trước khi gửi ảnh lên backend, frontend sử dụng mô hình nhận diện khuôn mặt (*face_detector*) để kiểm tra số lượng khuôn mặt trong ảnh.

- **Kiểm tra điều kiện hợp lệ ngay trên frontend:**

Mô hình nhận diện khuôn mặt chạy ngay trên frontend nhằm đảm bảo ảnh chứa đúng một người trước khi gửi lên backend. Nếu ảnh không có người hoặc có nhiều hơn một người, frontend sẽ thông báo lỗi và yêu cầu người dùng thử lại mà không cần gửi ảnh lên backend.

- **Xử lý tại backend:**

Khi ảnh được frontend xác nhận hợp lệ và gửi lên, backend sẽ nhận ảnh thông qua các API chuyên biệt và tiến hành gửi ảnh này đến các mô hình AI phân tích đặc điểm da.

- **Phân tích da bằng các mô hình AI:**

Ảnh hợp lệ sẽ lần lượt được gửi đến ba mô hình AI chuyên biệt:

- Mô hình phát hiện mụn (acne detection)
- Mô hình phát hiện nám (dark spot detection)
- Mô hình phát hiện nếp nhăn (wrinkle detection)

Các mô hình này hoạt động độc lập, thường được triển khai dưới dạng các dịch vụ con (microservices), có thể xử lý song song để tối ưu thời gian phản hồi.

- **Tổng hợp kết quả và lưu trữ:**

Sau khi nhận được kết quả phân tích từ các mô hình AI, backend sẽ tiến hành tổng hợp, chuẩn hóa định dạng, và lưu trữ kết quả vào cơ sở dữ liệu (MongoDB). Dữ liệu lưu bao gồm ID người dùng, thời gian, đường dẫn ảnh, và thông tin phân tích chi tiết.

- **Hiển thị kết quả cho người dùng:**

Cuối cùng, frontend gửi yêu cầu để lấy kết quả phân tích và hiển thị cho người dùng dưới dạng trực quan. Giao diện kết quả cho thấy từng loại phân tích (mụn, nám, nếp nhăn) với các chỉ số đánh giá, kèm hình ảnh minh họa nếu cần.

4. Thiết kế thành phần

4.1. Thiết kế API

Hệ thống Backend API được tổ chức theo kiến trúc mô-đun, đảm bảo khả năng mở rộng và dễ bảo trì. Mỗi chức năng chính được phân chia thành các module riêng biệt, bao gồm định tuyến (routes), xử lý nghiệp vụ (services), điều phối logic (controllers), và các tiện ích dùng chung (utils). Toàn bộ API được triển khai theo chuẩn RESTful, đóng vai trò trung gian giữa frontend và các dịch vụ xử lý AI cũng như cơ sở dữ liệu.

Các chức năng của hệ thống được phân chia rõ ràng thành từng nhóm API

- **auth_routes.py:**

Xử lý các chức năng xác thực người dùng như:

- Đăng ký (*/auth/register*)
- Đăng nhập (*/auth/login*)
- Đăng xuất (*/auth/logout*)

- **user_routes.py:**

Dành cho người dùng đã đăng nhập, bao gồm:

- Xem hồ sơ cá nhân (*/user/profile*)
- Cập nhật thông tin cá nhân (*/user/update_profile*)
- Xác minh mật khẩu (*/user/verify_password*)

- **admin_routes.py:**

Chỉ admin mới truy cập được, bao gồm:

- Xem danh sách người dùng (*/admin/users*)
- Thêm người dùng (*/admin/users/add*)
- Xóa người dùng (*/admin/users/remove/<user_id>*)
- Cập nhật vai trò người dùng (*/admin/users/update_role/<user_id>*)

- **stream_routes.py:**

Xử lý các yêu cầu thời gian thực qua WebSocket, bao gồm:

- Nhận ảnh từ client
- Gửi ảnh đến các mô hình AI xử lý song song
- Trả kết quả phân tích về cho client theo thời gian thực
- Tự động ngắt kết nối khi tất cả mô hình hoàn tất dự đoán

- **analyze_routes.py**

Là nhóm API chuyên gọi đến 3 mô hình phân tích da chính, hỗ trợ:

- Đánh giá tình trạng da dựa trên ảnh người dùng gửi lên

- Trả về kết quả phân tích chi tiết giúp người dùng hiểu rõ hơn về sức khỏe làn da

Cách API hoạt động

Mỗi khi frontend gửi yêu cầu, hệ thống sẽ xử lý qua các bước:

- **Route (định tuyến):**
Hệ thống xác định đúng đường dẫn URL và phương thức HTTP (GET, POST, PUT, DELETE) tương ứng với nhóm API và chức năng cần gọi.
- **Controller (bộ điều phối):**
 - Controller nhận dữ liệu vào, kiểm tra tính hợp lệ, quyền truy cập.
 - Gọi các hàm xử lý nghiệp vụ (services) tương ứng để thực hiện các thao tác cụ thể như truy vấn cơ sở dữ liệu, xử lý dữ liệu đầu vào, gọi mô hình AI,...
 - Controller cũng chịu trách nhiệm chuẩn bị dữ liệu kết quả trả về.
- **Utils / DB connector:**
 - Các hàm tiện ích hỗ trợ thao tác với cơ sở dữ liệu MongoDB, bao gồm truy vấn, thêm, sửa, xóa dữ liệu.
 - Xử lý các thao tác phụ trợ như mã hóa mật khẩu, xác thực token, kiểm tra quyền truy cập người dùng.
- **Trả phản hồi:**
 - Kết quả xử lý được trả về frontend dưới dạng JSON chuẩn, bao gồm dữ liệu cần thiết hoặc thông báo lỗi rõ ràng nếu có.
 - Hệ thống đảm bảo các phản hồi này nhất quán, dễ hiểu để frontend dễ dàng xử lý và hiển thị.

Ưu điểm thiết kế

- **Rõ ràng, dễ mở rộng:** Thêm chức năng mới chỉ cần tạo thêm route và controller tương ứng.
- **Bảo mật phân quyền:** Kiểm tra quyền (admin/user) ở mỗi request, tránh truy cập trái phép.
- **Tách biệt logic:** Xử lý nghiệp vụ được tổ chức riêng, dễ kiểm tra, bảo trì.

4.2. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ, với các bảng phản ánh các thực thể chính trong hệ thống như người dùng, lịch sử phân tích, kết quả xử lý ảnh và phân quyền. Thiết kế này bảo đảm khả năng mở rộng, toàn vẹn dữ liệu và hỗ trợ hiệu quả cho các truy vấn phân tích và thống kê trong tương lai.

Hệ thống sử dụng **MongoDB** làm cơ sở dữ liệu để lưu trữ thông tin người dùng. Dữ liệu được tổ chức dưới dạng document trong một collection duy nhất là *users*. Mỗi document trong *users* đại diện cho một người dùng, bao gồm các trường sau:

- **_id**: ID mặc định của MongoDB, được tạo tự động.
- **username**: Tên đăng nhập của người dùng, cần đảm bảo duy nhất.
- **email**: Địa chỉ email liên kết với tài khoản, dùng để khôi phục mật khẩu hoặc gửi thông báo.
- **password**: Mật khẩu đã được mã hóa (hash) để đảm bảo an toàn.
- **role**: Vai trò của người dùng trong hệ thống, ví dụ: *user* (người dùng thông thường), *admin* (quản trị viên).

Collection này giúp hệ thống quản lý người dùng, hỗ trợ đăng nhập, phân quyền và bảo mật.

Thiết kế cơ sở dữ liệu đơn giản, tập trung vào người dùng, giúp hệ thống dễ mở rộng và tích hợp các chức năng quản lý tài khoản, phân quyền và lưu trữ kết quả trong các giai đoạn phát triển tiếp theo.

4.3. Thiết kế giao diện người dùng

Hệ thống giao diện người dùng được thiết kế đơn giản, thân thiện, dễ sử dụng cho cả người mới. Giao diện được xây dựng với HTML/CSS/JavaScript thuần, hỗ trợ bởi một số thư viện xử lý nhận diện khuôn mặt phía client. Toàn bộ phần frontend hoạt động trong trình duyệt và kết nối đến backend thông qua các API.

Cấu trúc giao diện và tổ chức mã nguồn

- Giao diện được xây dựng dựa trên các trang HTML tĩnh, kèm theo các tập tin CSS và JavaScript riêng biệt để dễ dàng quản lý và bảo trì.
- Các thành phần chính được tổ chức trong các thư mục riêng biệt như:

- *html/*: Chứa các trang web chính như trang đăng nhập, trang hồ sơ, trang quét da, trang lịch sử.
- *css/*: Chứa các tập tin định dạng, giúp đồng bộ màu sắc, kiểu chữ và bố cục giữa các trang.
- *javascript/*: Chứa các tập tin xử lý logic phía client, bao gồm quản lý tương tác người dùng, gọi API backend và xử lý kết quả trả về.
- *pictures/*: Lưu trữ các ảnh minh họa hỗ trợ cho giao diện.

Thiết kế trải nghiệm người dùng (UX)

- Các trang được thiết kế rõ ràng với các chức năng chính phân chia hợp lý: đăng nhập, xem và chỉnh sửa thông tin cá nhân, thực hiện quét da, xem lịch sử quét.
- Giao diện sử dụng màu sắc nhẹ nhàng, dễ nhìn, giúp người dùng tập trung vào nội dung chính. Font chữ và kích thước được chọn để đảm bảo khả năng đọc tốt trên các thiết bị khác nhau.
- Tính năng phản hồi tức thì được ưu tiên, ví dụ khi đăng nhập hoặc cập nhật thông tin, hệ thống sẽ thông báo rõ trạng thái thành công hoặc lỗi.
- Thiết kế đảm bảo tính responsive, giao diện tự điều chỉnh phù hợp với các kích thước màn hình khác nhau, từ máy tính bàn đến điện thoại thông minh.

Luồng tương tác chính của người dùng

- Người dùng bắt đầu bằng việc truy cập trang đăng nhập (*login.html*), nhập thông tin và xác thực qua backend.
- Sau khi đăng nhập thành công, người dùng có thể truy cập trang hồ sơ (*profile.html*) để xem và cập nhật thông tin cá nhân.
- Trang quét da (*scan.html*) cho phép người dùng kích hoạt camera, thực hiện nhận diện khuôn mặt và phân tích tình trạng da.
- Người dùng cũng có thể truy cập trang lịch sử (*history.html*) để xem lại các kết quả quét da đã lưu.
- Cuối cùng, người dùng có thể đăng xuất để bảo vệ an toàn tài khoản.

Công nghệ và công cụ sử dụng

- Giao diện sử dụng HTML, CSS và JavaScript thuần, không phụ thuộc nhiều vào thư viện lớn, giúp giảm độ phức tạp và tăng tốc độ tải trang.

- Mô hình nhận diện khuôn mặt sử dụng thư viện *face-api.js*, tải và chạy trực tiếp trong trình duyệt, tránh gửi ảnh nhạy cảm lên server.
- Hệ thống frontend được Docker hóa với Nginx làm web server, tối ưu việc phục vụ tệp tĩnh và điều phối API requests đến backend.
- Tập tin *Dockerfile* và cấu hình *nginx.conf* đảm bảo dễ dàng triển khai và vận hành trong môi trường container.

5. Thiết kế chi tiết mô hình AI

Hệ thống AI Skinsight sử dụng ba mô hình chuyên biệt để thực hiện phân tích hình ảnh khuôn mặt và phát hiện ba loại vấn đề về da: mụn, nám và nếp nhăn. Mỗi mô hình đảm nhiệm một nhiệm vụ phân loại độc lập, được huấn luyện với kiến trúc tối ưu cho loại đặc trưng da tương ứng, từ đó giúp nâng cao độ chính xác và tính chuyên biệt hóa của hệ thống.

Mục tiêu chính của thiết kế mô hình AI là đảm bảo khả năng phân tích ảnh chính xác, nhanh chóng và có thể triển khai hiệu quả trong môi trường thực tế, nơi yêu cầu phản hồi theo thời gian thực và hoạt động song song. Bên cạnh đó, các mô hình phải đảm bảo tương thích với hệ thống xử lý ảnh đầu vào, hỗ trợ chuẩn hóa dữ liệu, kiểm tra tính hợp lệ khuôn mặt và dễ dàng chuyển đổi sang định dạng triển khai.

5.1. Mô hình phát hiện mụn

Mô hình phát hiện mụn sử dụng kiến trúc YOLOv8n (YOLO version 8 - Nano), một mô hình nhận diện đối tượng thời gian thực được phát triển bởi Ultralytics. Đây là phiên bản nhẹ nhất trong dòng YOLOv8, được thiết kế để cân bằng giữa độ chính xác và tốc độ xử lý, đặc biệt phù hợp với các hệ thống cần inference nhanh trên thiết bị có tài nguyên giới hạn hoặc triển khai dạng dịch vụ trực tuyến.

5.1.1. Kiến trúc mô hình

Cấu trúc tổng quát của YOLOv8n

YOLOv8n có ba thành phần chính trong pipeline xử lý ảnh đầu vào:

- **Backbone (Feature Extractor):**

Thành phần này sử dụng mạng CNN hiệu quả để trích xuất đặc trưng từ ảnh đầu vào. YOLOv8 áp dụng C2f (Concatenate-to-fuse) block, một biến thể tối ưu hơn của CSP (Cross Stage Partial), để giảm độ phức tạp tính toán trong khi vẫn giữ được khả năng trích xuất đặc trưng sâu.

- **Neck (Feature Fusion):**

Neck là nơi kết hợp các đặc trưng ở nhiều cấp độ khác nhau, sử dụng cấu trúc FPN + PAN (Feature Pyramid Network và Path Aggregation Network). Điều này giúp mô hình có khả năng phát hiện đối tượng ở

nhiều kích thước khác nhau, phù hợp với đặc điểm của mụn vốn có kích thước nhỏ và thay đổi theo vùng da.

- **Head (Prediction):**

Phần đầu ra của mô hình bao gồm các tensor dự đoán:

- Vị trí bounding box (x, y, width, height)
- Confidence score (mức độ chắc chắn có mụn)
- Lớp nhãn (ở đây chỉ có một lớp: “Acne”)

5.1.2. Dữ liệu huấn luyện

Tập dữ liệu được sử dụng có định dạng YOLOv8 và bao gồm ba tập: train, valid, và test. Dữ liệu chứa các ảnh khuôn mặt người thật với nhãn "Acne" đánh dấu các vùng bị mụn bằng bounding box. Cấu trúc thư mục được quy định trong tệp YAML như sau:

```
path: /kaggle/input/acne-dataset-in-yolov8-format/data-2
train: train/images
val: valid/images
test: test/images
names: ['Acne']
```

Tập cấu hình này được lưu lại dưới tên ‘*acne.yaml*’ và dùng làm đầu vào cho quá trình huấn luyện.

5.1.3. Cấu hình huấn luyện

Quá trình huấn luyện được thực hiện với các tham số như sau:

- **Mô hình gốc:** yolov8n.pt (được pretrained trên COCO)
- **Epochs:** 100
- **Kích thước ảnh (imgsz):** 640x640 pixel
- **Batch size:** 16
- **Optimizer:** Adam
- **Learning rate khởi tạo:** 0.0007
- **Weight decay:** 0.0009
- **Patience (early stopping):** 20 epoch không cải thiện

5.2. Mô hình phát hiện nám

Mô hình sử dụng kiến trúc Faster R-CNN với backbone ResNet-50 và FPN (Feature Pyramid Network), một trong những kiến trúc mạnh mẽ và phổ biến nhất trong lĩnh vực phát hiện đối tượng. Đây là mô hình có khả năng đạt độ chính xác cao, đặc biệt phù hợp cho các đối tượng nhỏ, mờ như các đốm nám.

5.2.1. Kiến trúc mô hình

Cấu trúc chính của mô hình gồm:

- **Backbone – ResNet-50 + FPN:**
ResNet-50 được sử dụng để trích xuất đặc trưng từ ảnh đầu vào. Các đặc trưng ở nhiều cấp độ được kết hợp thông qua FPN, giúp tăng khả năng phát hiện các đốm nám có kích thước và độ tương phản khác nhau.
- **Region Proposal Network (RPN):**
Thành phần này sinh ra các vùng đề xuất có khả năng chứa đối tượng. Đây là điểm mạnh của Faster R-CNN giúp tách biệt việc "đề xuất vùng" và "phân loại".
- **ROI Pooling + Classification Head:**
Các proposal được cắt và đưa qua các tầng fully connected để phân loại và hồi quy bounding box. Với bài toán này, mô hình chỉ cần phân biệt giữa nám và background.

5.2.2. Dữ liệu huấn luyện

Dữ liệu được gán nhãn theo định dạng COCO, bao gồm:

- Ảnh khuôn mặt người thật với các vùng nám được annotate dưới dạng bounding box.
- Hai tập dữ liệu: tập huấn luyện (train) và tập kiểm thử (valid) riêng biệt.

Các annotation được lưu trong tệp JSON theo chuẩn COCO, chứa thông tin bounding box và lớp nhãn.

5.2.3. Cấu hình huấn luyện

- **Mô hình:** `fasterrcnn_resnet50_fpn(pretrained=True)`
- **Số lớp phân loại:** 2 (1 lớp nám + 1 lớp nền)
- **Thiết bị:** Tự động sử dụng GPU nếu khả dụng (`torch.device("cuda")`)
- **Bộ tối ưu hóa:** SGD
 - Learning rate: 0.005
 - Momentum: 0.9
 - Weight decay: 0.0005
- **Số epoch:** 10
- **Batch size:** 4
- **Loss function:** Tự động nội suy từ tổn thất RPN và đầu ra của ROI Heads (sẵn có trong FasterRCNN)

5.3. Mô hình phát hiện nếp nhăn

Hệ thống sử dụng mô hình U-Net để phân đoạn vùng có nếp nhăn trong ảnh khuôn mặt. U-Net là một mạng nơ-ron tích chập đặc biệt phổ biến trong bài toán phân đoạn ảnh y sinh và hình ảnh y tế

5.3.1. Kiến trúc mô hình

Mô hình gồm

- **Encoder (Contracting path):** 4 khối *conv_block* với các số filters lần lượt là 64, 128, 256, 512. Mỗi khối gồm 2 lớp *Conv2D* + *BatchNorm*, sau đó là *MaxPooling* để giảm kích thước.
- **Bottleneck:** Một khối *conv_block* với 1024 filters giúp nắm bắt thông tin trừu tượng nhất.
- **Decoder (Expanding path):** Gồm 4 khối sử dụng *Conv2DTranspose* để tăng kích thước và kết hợp (concatenate) với các feature map từ encoder tương ứng theo kiến trúc skip-connection.
- **Output Layer:** Một lớp *Conv2D(1x1)* với activation *sigmoid* để dự đoán mặt nạ nhị phân xác định vùng nếp nhăn.

Hàm mất mát (Loss Function)

- Kết hợp giữa *Binary Crossentropy* và *Dice Loss*, nhằm tối ưu vừa pixel-wise accuracy, vừa độ chồng lấp của vùng phân đoạn.

Metric đánh giá: *Dice Coefficient* — đo mức độ trùng khớp giữa mask dự đoán và mask thật.

5.3.2. Dữ liệu huấn luyện

Nguồn dữ liệu

Ảnh khuôn mặt và mặt nạ thủ công được cung cấp trong hai thư mục riêng biệt:

- images/: ảnh gốc khuôn mặt
- masks/: ảnh mask chứa vùng đánh dấu nếp nhăn

Tiền xử lý và xử lý mask

- Resize ảnh và mask về kích thước cố định (256, 256)
- Chuẩn hóa ảnh về dải giá trị [0, 1]
- Chuyển mask sang nhị phân, gồm:
 - Pixel trắng và pixel xanh lá được gán là 1 (wrinkle)
 - Các pixel còn lại là 0 (non-wrinkle)

Tăng cường dữ liệu (Data Augmentation)

Sử dụng Albumentations để tăng sự đa dạng của dữ liệu huấn luyện bằng cách áp dụng:

- Flip ngang/dọc, xoay ngẫu nhiên
- Dịch chuyển, co giãn, biến dạng đàn hồi
- Làm mờ Gaussian và điều chỉnh độ sáng/tương phản

Sau augmentation, số mẫu được gấp đôi, giúp giảm overfitting và tăng khả năng khái quát hóa của mô hình.

5.3.3. Cấu hình huấn luyện

- **Mô hình:** U-Net tùy chỉnh
- **Kích thước ảnh đầu vào:** (256, 256, 3)
Loss function: combined_bce_dice_loss (BCE + Dice Loss)
- **Optimizer:** Adam
- **Metric:** dice_coef
- **Batch size:** 16
- **Số epoch:** 10

5.4. Chuyển đổi mô hình sang định dạng ONNX

Sau khi huấn luyện xong, ba mô hình nhận diện đặc điểm khuôn mặt: phát hiện mụn, phát hiện nám và phát hiện nếp nhăn đều được chuyển đổi sang định dạng ONNX (Open Neural Network Exchange) để phục vụ giai đoạn triển khai.

Lý do sử dụng ONNX

- ONNX là một định dạng mô hình trung gian, cho phép các mô hình huấn luyện từ các framework khác nhau (TensorFlow, PyTorch, Keras, ...) có thể được chuyển đổi và chạy nhất quán trên cùng một nền tảng.
- **Đồng bộ hóa hệ thống:** Cả 3 mô hình gốc được xây dựng bằng 2 framework khác nhau, nếu giữ nguyên sẽ gây khó khăn khi tích hợp trong Docker. Chuyển sang ONNX giúp chuẩn hóa luồng xử lý.
- **Hiệu suất cao:** ONNX có thể được tối ưu với các backend như ONNX Runtime, TensorRT, hoặc OpenVINO, giúp tăng tốc độ inference, đặc biệt hữu ích khi chạy song song trong môi trường đa mô hình.
- **Triển khai linh hoạt:** Có thể dễ dàng triển khai trên nhiều nền tảng khác nhau (CPU, GPU, edge devices) mà không phụ thuộc vào framework gốc.

Lợi ích

- **Giảm độ trễ** khi chạy đồng thời 3 mô hình sau khi cắt khuôn mặt từ ảnh.
- **Tăng khả năng mở rộng**, dễ dàng thay thế hoặc cập nhật mô hình mà không ảnh hưởng tới pipeline chính.
- **Đơn giản hóa việc đóng gói và triển khai** trong Docker với mỗi mô hình là một container độc lập chạy ONNX Runtime, giao tiếp qua API nội bộ).

6. Hướng phát triển trong tương lai

Để đảm bảo hệ thống luôn đáp ứng được nhu cầu ngày càng cao của người dùng và tận dụng hiệu quả công nghệ mới, nhóm đề xuất các hướng phát triển sau:

- **Mở rộng các mô hình AI phân tích da**

Trong tương lai, có thể bổ sung thêm các mô hình mới như phát hiện tàn nhang, độ ẩm da, hoặc các vấn đề da liễu khác nhằm nâng cao độ chính xác và đa dạng của kết quả phân tích.

- **Tối ưu trải nghiệm người dùng trên frontend**

Nâng cấp giao diện người dùng với các tính năng tương tác trực quan hơn, ví dụ như cung cấp biểu đồ lịch sử sức khỏe da theo thời gian, và đưa ra các gợi ý chăm sóc cá nhân hóa dựa trên kết quả phân tích.

- **Cải tiến hiệu suất và khả năng mở rộng backend**

Áp dụng các kỹ thuật tối ưu như caching, load balancing và microservices để đảm bảo hệ thống xử lý nhanh và ổn định khi số lượng người dùng tăng cao.

- **Bảo mật và bảo vệ dữ liệu người dùng**

Tăng cường các biện pháp bảo mật như mã hóa dữ liệu nhạy cảm, xác thực đa yếu tố (MFA), và quản lý quyền truy cập chặt chẽ để bảo vệ thông tin cá nhân cũng như kết quả phân tích của người dùng.

- **Phát triển API mở cho các ứng dụng bên ngoài**

Xây dựng API chuẩn để các nhà phát triển bên thứ ba có thể tích hợp dịch vụ phân tích da vào ứng dụng hoặc nền tảng riêng, từ đó mở rộng hệ sinh thái và tăng khả năng ứng dụng của hệ thống.

- **Nghiên cứu và áp dụng công nghệ mới**

Theo dõi xu hướng phát triển AI, học máy và xử lý ảnh để áp dụng các mô hình tiên tiến hơn, cải thiện độ chính xác phân tích và giảm thiểu thời gian xử lý.

Với các hướng phát triển này, hệ thống sẽ không ngừng được hoàn thiện, đáp ứng tốt hơn nhu cầu người dùng và tăng cường giá trị sử dụng trong tương lai.

7. Kết luận

Tài liệu thiết kế hệ thống AI Skinsight đã miêu tả đầy đủ kỹ thuật tổng thể của một nền tảng phân tích da thông minh, từ kiến trúc hệ thống, tổ chức mã nguồn, thiết kế API, đến việc tích hợp các mô hình AI chuyên biệt và hệ quản trị cơ sở dữ liệu. Việc áp dụng kiến trúc microservices, kết hợp cùng Docker và nguyên tắc tách biệt frontend - backend, không chỉ đảm bảo tính mô-đun và hiệu quả triển khai, mà còn mở ra khả năng mở rộng linh hoạt cho hệ thống trong tương lai.

Hệ thống được thiết kế với tinh thần phân tách trách nhiệm rõ ràng: từng mô hình AI đảm nhiệm một chức năng độc lập (mụn, nám, nếp nhăn), các API đảm bảo luồng giao tiếp ổn định, trong khi tầng giao diện mang đến trải nghiệm người dùng mượt mà, trực quan. Tư duy này không chỉ tối ưu quy trình phát triển mà còn góp phần nâng cao khả năng kiểm thử, bảo trì và thích ứng với sự thay đổi của công nghệ hoặc nhu cầu thị trường.

Dù không thay thế vai trò của bác sĩ da liễu, AI Skinsight hướng đến vai trò một công cụ hỗ trợ mang tính tham khảo cao, giúp người dùng quan tâm đến làn da có thêm một kênh phân tích tin cậy, dễ sử dụng và có giá trị thực tiễn. Trên nền tảng kỹ thuật đã được chuẩn hóa, hệ thống hoàn toàn sẵn sàng để bước tiếp, phát triển thêm các mô hình phân tích chuyên sâu, tích hợp hệ thống theo dõi tiến trình điều trị, hoặc mở rộng ứng dụng vào các dịch vụ chăm sóc sắc đẹp chuyên nghiệp.